

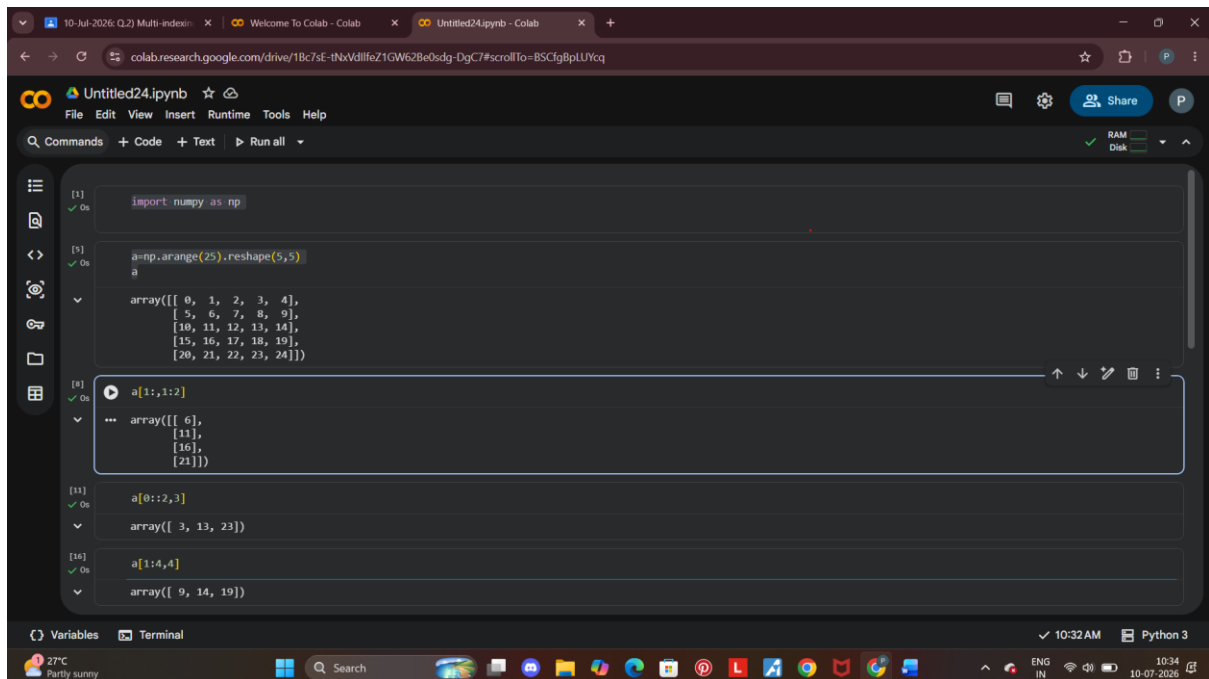
Code:

```
import numpy as np
```

```
a=np.arange(25).reshape(5,5)
```

```
a
```

Output:



The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
[1]: import numpy as np

[2]: a=np.arange(25).reshape(5,5)
a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])
```

The output of the code is displayed below the code cell:

```
[3]: a[1:,1:2]
array([[ 6],
       [11],
       [16],
       [21]])
```

The next code cell shows the output of `a[0::2,3]`:

```
[11]: a[0::2,3]
array([ 3, 13, 23])
```

The final code cell shows the output of `a[1::4,4]`:

```
[16]: a[1::4,4]
array([ 9, 14, 19])
```

The bottom of the screenshot shows the Windows taskbar with the date and time: 10:34 10-07-2026.

Q.1

Code:

`a[0:,0::2]`

Output:

The screenshot shows a Google Colab notebook with the following code and output:

```
array([ 9, 14, 19])
```

```
[23] a[3:, :3]
```

```
array([[15, 16, 17],
```

```
       [20, 21, 22]])
```

```
[24] a[0:, 0::2]
```

```
array([[ 0,  2,  4],
```

```
       [ 5,  7,  9],
```

```
       [10, 12, 14],
```

```
       [15, 17, 19],
```

```
       [20, 22, 24]])
```

```
[25] a[1::2, 1::2]
```

```
array([[ 6,  8],
```

```
       [16, 18]])
```

The output of the indexing operation `a[0:, 0::2]` is a 1D array: `array([ 0, 2, 4])`.

Insight:

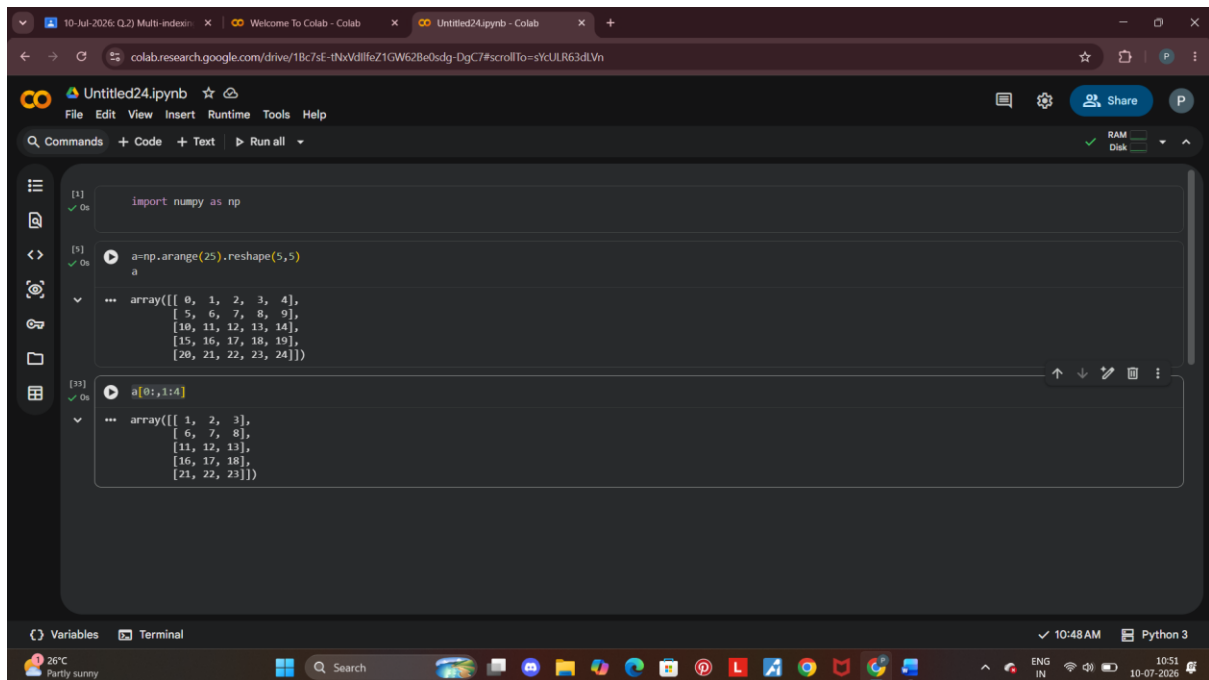
- In the main matrix was of 5x5, in above code of multi-indexing it skipped one column so we only got 0,2,4.

Q.2

Code:

```
a[0:,1:4]
```

Output:



```
[1]: import numpy as np

[5]: a=np.arange(25).reshape(5,5)
a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])

[11]: a[0:,1:4]
array([[ 1,  2,  3],
       [ 6,  7,  8],
       [11, 12, 13],
       [16, 17, 18],
       [21, 22, 23]])
```

Insight:

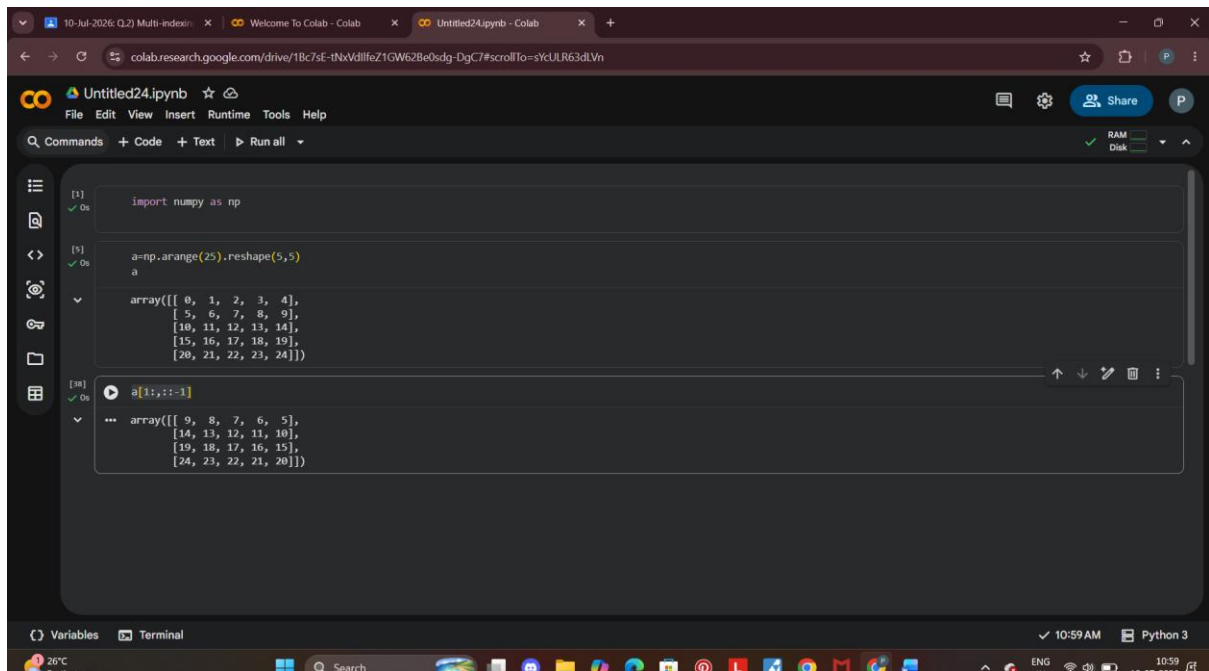
- In the main matrix was of 5x5, in above code of multi-indexing it skipped the top left corner (0) and top right corner (1).

Q.3

Code:

```
a[1::-1]
```

Output:



```
[1]: import numpy as np

[5]: a=np.arange(25).reshape(5,5)
a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])

[10]: a[1::-1]
array([[ 9,  8,  7,  6,  5],
       [14, 13, 12, 11, 10],
       [19, 18, 17, 16, 15],
       [24, 23, 22, 21, 20]])
```

Insight:

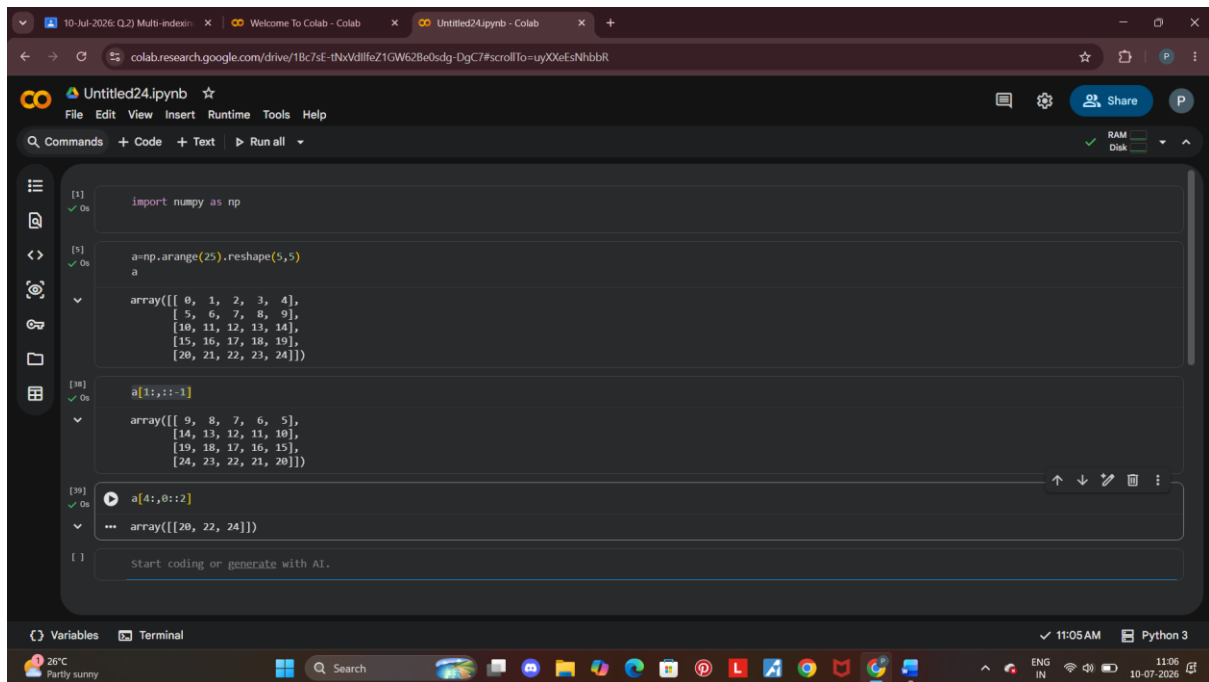
- In the main matrix was of 5x5, in above code of multi=indexing, the two colons represent the starting and ending position which is empty, by writing -1 it reverses the data.
- -1 is considered as the index position if last element in a list or array.
- So writing ::-1 prints the reversed array.

Q.4

Code:

```
a[4:,0::2]
```

Output:



```
[1] import numpy as np

[5] a=np.arange(25).reshape(5,5)
a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])

[18] a[1:,1:-1]
array([[ 9,  8,  7,  6,  5],
       [14, 13, 12, 11, 10],
       [19, 18, 17, 16, 15],
       [24, 23, 22, 21, 20]])

[39] a[4:,0::2]
array([[20, 22, 24]])

[ ] Start coding or generate with AI.
```

Insight:

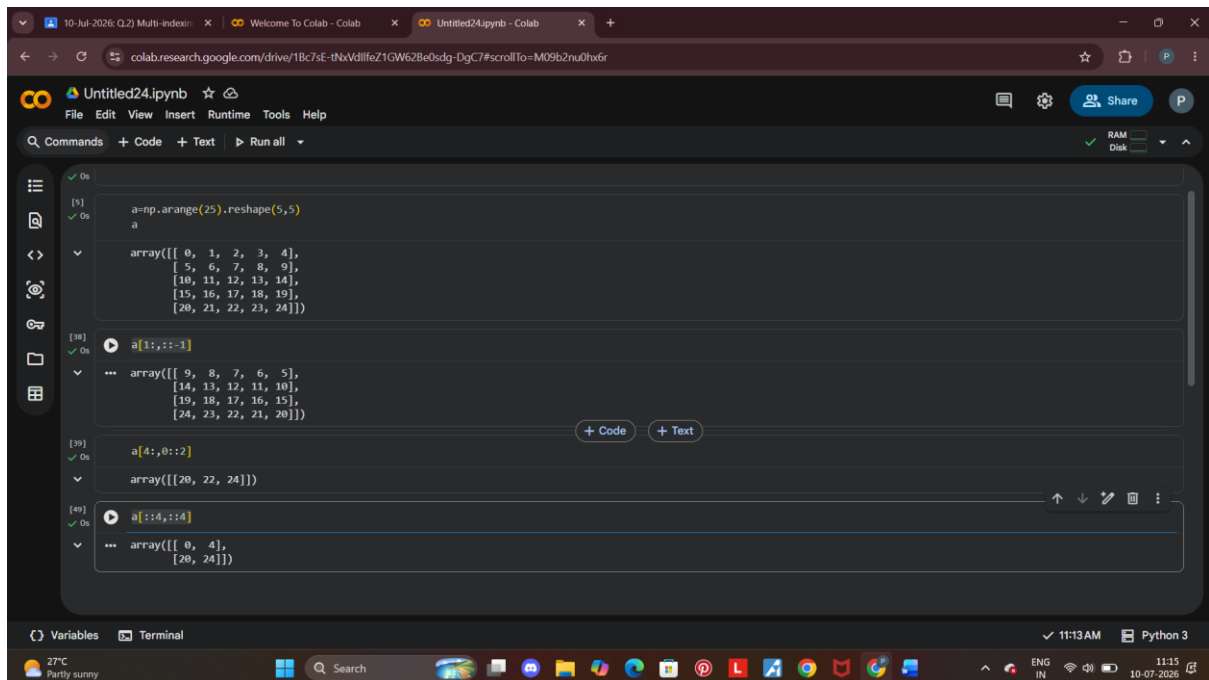
- In the main matrix was of 5x5, in above code of multi=indexing it started from row number 4 and skipped every second column.

Q.5

Code:

```
a[::4,::4]
```

Output:



```
[5]: a=np.arange(25).reshape(5,5)
a

array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])

[18]: a[1::1]

array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])

[39]: a[4:,0::2]

array([[20, 22],
       [24, 20]])

[49]: a[::4,::4]

array([[ 0,  4],
       [20, 24]])
```

Insight:

- In the main matrix was of 5x5, in above code of multi-indexing it started from row 0 and stopped at row 4 same with column.
- ::4 skipped all the 1,2,3 index from the matrix and fetched only the 0 and 4th index position elements.

